

Estados

O estado no React é um valor armazenado em memória que permite ao componente controlar algum aspecto da interface. Pode ser utilizado para controlar a exibição ou ocultação de um componente, mostrar valores dentro de um campo de entrada, entre outras funcionalidades.

É importante destacar que o estado é mantido apenas em memória e, portanto, se você recarregar a página ou o componente for desmontado, o valor do estado será perdido.

Quando usar?

Você deve usar o estado no React quando precisar **controlar** e **atualizar** dados que podem mudar ao longo da execução do seu componente. Aqui estão algumas situações em que é comum e recomendado utilizar o estado:

- Se o componente precisa responder a ações do usuário, como cliques em botões, movimentos do mouse, etc;
- Quando você deseja lidar com campos de entrada, como caixas de texto, seletores ou caixas de seleção;
- Se você precisa mostrar ou esconder elementos com base em certas condições.

Como utilizar?

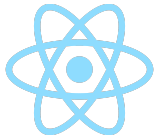
Para utilizar estados em um componente React, você precisa usar o hook [useState](#).

```
import { useState } from "react";

export function MyComponent() {
  const [state, setState] = useState(null);
};
```

Ao utilizar o `useState`, você recebe um array com 2 valores, sendo o primeiro o valor atual do seu estado, e o segundo, uma função que atualiza o estado. Você pode chamar esses valores como desejar.

Por convenção, é comum nomear o estado e sua função de atualização seguindo o padrão "valor" e "setValor", substituindo "valor" por um nome que faça sentido para o seu componente.



Vamos à um exemplo prático:

```
import React, { useState } from 'react';

export function Counter() {
  const [count, setCount] = useState(0);

  function incrementCounter(){
    setCount(count + 1);
  };

  return (
    <div>
      <p>Contador: {count}</p>
      <button onClick={incrementCounter}>Increment</button>
    </div>
  );
};
```

Neste exemplo, estamos utilizando o hook `useState` para criar um estado chamado `count` com um valor inicial de 0. O `useState` retorna uma array com duas posições: o valor atual do estado e uma função para atualizar esse valor (`setCount`).

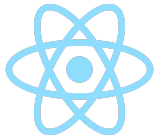
Dentro do componente, exibimos o valor do estado `count` e adicionamos um botão que chama a função `incrementCounter` quando clicado. Essa função utiliza `setCount` para atualizar o valor do estado, adicionando 1 ao valor atual.

Quando o estado é atualizado usando `setCount`, o React re-renderiza o componente com o novo valor do estado, atualizando a exibição na tela.

Como já iniciamos o estado com um “0”, não será necessário de fato definir a tipagem, pois já fica subentendido ao TypeScript que aquele estado somente aceitará números. No entanto, pode ser uma boa prática adicionar a tipagem mesmo quando o TypeScript é capaz de inferir o tipo corretamente. Isso ajuda a tornar o código mais legível e documentado, além de fornecer um ponto de referência claro para outros desenvolvedores que possam trabalhar no código.

Para fazer o uso da tipagem, siga a sintaxe abaixo:

```
const [count, setCount] = useState<number>(0);
```



Nesse caso, a tipagem do estado `count` é definida utilizando a sintaxe `useState<number>`. O tipo `number` dentro dos ângulos `< >` indica que o estado `count` será do tipo `number`.

É importante lembrar que os estados devem ser tratados como imutáveis no React. Em vez de modificar diretamente o valor do estado, como `count++`, é recomendado utilizar a função de atualização fornecida pelo hook para garantir que o React atualize corretamente o estado e o componente seja re-renderizado adequadamente!